Brief description of the code.

The main code and auxiliary codes are developed in `FORTRAN` language. Starting with the auxiliary codes, there is the `gera_poly_EM34_v4.f90` code, which creates the base nodes for each region of the model and the measurement positions from the `blocos.in` input file. With the base nodes for each region, Tetgen is executed to generate the mesh. We also have the `gera_arestas.f90` code, responsible for creating a file with the relationship between each element and its edges, with information about elements, nodes, and edges from `TetGen` files.

`TetGen` is a program for generating tetrahedral meshes from three-dimensional polyhedral domains. Its purpose is to generate good quality tetrahedral meshes, suitable for various applications in scientific computing. It can be used as a standalone program or as a library component integrated with other software.

With the mesh constructed, the main code reads it and the conductive properties of the model. It then constructs the elementary matrices and identifies the elements that are part of the anomalous body to subsequently calculate the primary electric field, that is, calculate the electric field at each edge after factoring the global matrix by the `PARDISO` software. To then construct the B matrix that represents the influence of the source on the $A\mathbf{x} = \mathbf{B}$ system, the code was parallelized with `MPI` communication routines to reduce calculation time.

With the $A\mathbf{x} = \mathbf{B}$ system complete, `PARDISO` calculates the secondary magnetic field at each edge of the mesh in relation to each observation position. The edges of the elements created at the observation points are identified so that we can perform a linear combination of the secondary field values using the basis vector functions to calculate a secondary magnetic field value at the center of the element. This value is added to the primary magnetic field to obtain the total magnetic field at each measurement position.

1 Instructions

First step: Enter the model information for mesh generation. This input data is entered through the blocos.in file located in the Malha folder contained in the main 3DVHMD folder. The format of this input data is described in the blocos.in file attached in the examples folders and organized as shown in Figure 1. **Second step**: The `blocos.in` file

```
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
!!!model 1: 1 block immersed in half-space
!!
Background (1D)
1.d12    3000.d0        ! Resistivity and thickness of the air layer.
1                                 ! Number of layers, including basement.
500.d0  500.d0  3000.d0 ! Resistivities and thickness of each layer.

Blocks (3D)
1                    ! number of bodies.
50.d0  0.d0 8.d0     ! Coordinates of the center of the body.
20.d0  10.d0 10.d0   ! Block edge lengths (x,y,z).
10.d0  10.d0         ! Resistivities in the horizontal and vertical directions.

Profile
51                  ! Number of observation points
1.83d0   0.d0       ! Coordinates (x0,y0) of the starting point of the profile (x0=0.5*coil spacing).
101.83d0  0.d0      ! Coordinates (x,y) of the profile end point (x = length of profile + 0.5*coil spacing).
-0.1d0              ! Z coordinate of the profile line.
0.d0  0.d0  -0.1d0  ! transmitter coordinates.
1.d0                ! Dipole moment.
3.66d0  9800.d0     ! coil spacing and frequency
```

Figura 1: Preview of the input file `blocos.in`

will be read by the `gera_poly_EM34_v4.f90` code (located in the Malha folder), which is compiled as follows:

```
gfortran gera_poly_EM34_v4.f90 -o gera_poly
```

After running it (`./gera.poly blocos.in`), three files will be generated:

- `obs_points.dat`: shows the coordinates of all observation points. Used to visualize the measurement points.
- `blocos_vfe.info`: shows the coordinates of the boundaries of the 3D model containing the layers and blocks to be discretized.
- `blocos.poly`: provides information about the model's geometry, including nodes, faces, number of holes, and regions, with each region being associated with an index and potentially having different volume restrictions.

Third step: The `blocos.poly` file is used by TetGen to generate the tetrahedral mesh of the 3D model, considering the geometry and physical properties of each region. The following command was used to compile:

```
tetgen -pq1.2 -keaA blocos.poly
```

The following files are generated from this compilation:

- `blocos.1.node`: list of all nodes in the mesh.
- `blocos.1.ele`: list of all tetrahedra in the mesh, specifying which nodes form each element.
- `blocos.1.face`: list of the mesh contour faces, i.e., the external surfaces that delimit the model.
- `blocos.1.edge`: list of contour edges.
- `blocos.1.vtk`: file that can be opened in ParaView software to view the mesh.

Step 4‘: These files created by tetgen will be input files for the `gera_arestas.f90` code (located in the Malha folder), which is compiled as follows:

```
gfortran gera_arestas.f90 -o gera_arestas
```

After execution (`./gera_arestas blocos`), two files are generated:

- `blocos.1.nedge`: contains the total number of elements and, for each element, the global indices of its 6 edges.
- `blocos_vfe.front`: contains the total number of boundary edges and the indices of the edges located on the surfaces of the domain.

The Malha folder already contains the executable files for the programs `gera_poly_EM34_v4.f90` and `gera_arestas.f90`, as well as the TetGen software compiler. Therefore, if no changes have been made to the files, it is not necessary to recompile according to steps 2, 3, and 4. Simply run the `gera_malha.sh` script (`./gera_malha.sh blocos`) located in the Malha folder. The files containing the mesh information will be created in the Mesh folder to

```
#!/bin/bash
modelo="$1"

extin=".in"
arq_in="$modelo$extin"

if [ -f "$arq_in" ]
then
    ./gera_poly $arq_in
else
    echo "Arquivo $arq_in não existe!"
    exit
fi

extpoly=".poly"
arq_poly="$modelo$extpoly"
tetgen -pq1.25 -keaA $arq_poly

./gera_arestas $modelo
```

Figura 2: Preview of the file `gera_malha.sh`

be read by the main program. Figure 3 shows the mesh files. After generating the mesh, we suggest viewing it. To do so, we use ParaView software, which will need to read the

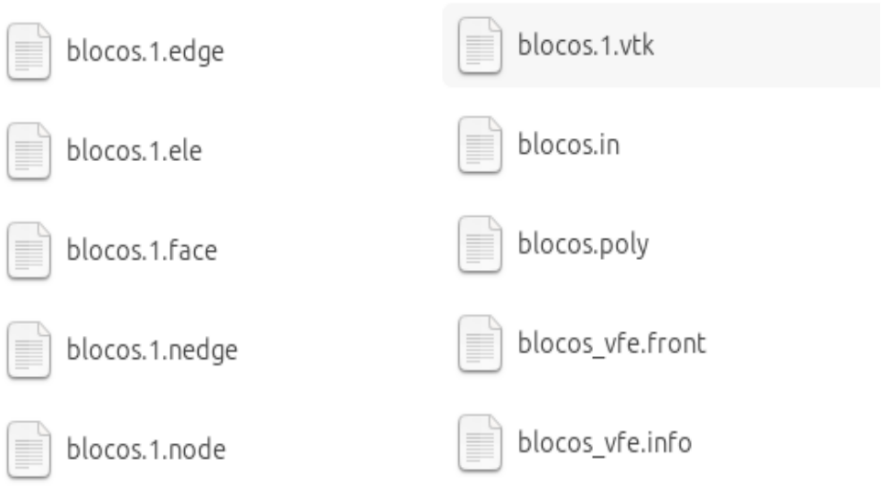

Figura 3: Preview of the files generated after executing the `gera_malha.sh` file.

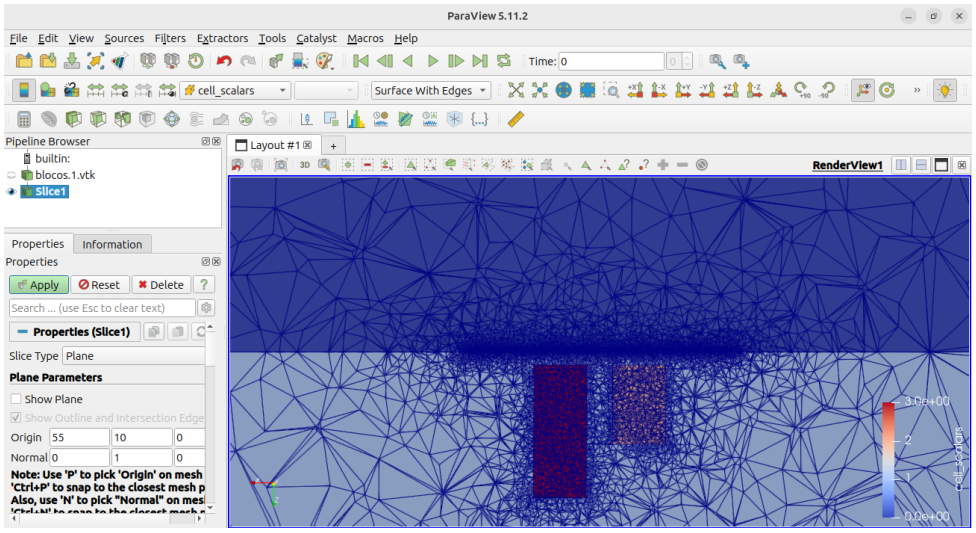

Figura 4: Visualization of the mesh generated by the file `blocos.1.vtk`

`blocos.1.vtk` file (see Figure 4). Now, if everything is as the user wants, compile the main program (`mpiifx -lmkl_rt hvmdmpi.f90 -o hvmd`) in the main `3DVHMD` folder, if you have made any changes to the code. If not, the executable is already created in the folder. Before running the generated executable, you must specify the number of cores available for `PARDISO` to use during factorization and secondary field calculation with the following command (export `OMP_NUM_THREADS=x`). This number will depend on the available computing resources. Once this is done, we can run the hvmd file with the following command (`time mpirun -np nx ./hvmd Mesh/blocks`), where nx is the number of processes involved in parallelization (Figure 5). The code creates two output files, hy.dat and hz.dat, with the positions of the measurements and the total field values of the Hy components for the HMD and Hz for the VMD.

```
a/3DVHMD$ mpiifx -lmkl_rt hvmdmpi.f90 -o hvmd
a/3DVHMD$ export OMP_NUM_THREADS=4
a/3DVHMD$ time mpirun -np 4 ./hvmd Malha/blocos
```